

CyberLab on AWS Academy Learner Lab: Step-by-Step Runbook

Sep 27, 2026 · @SaaId

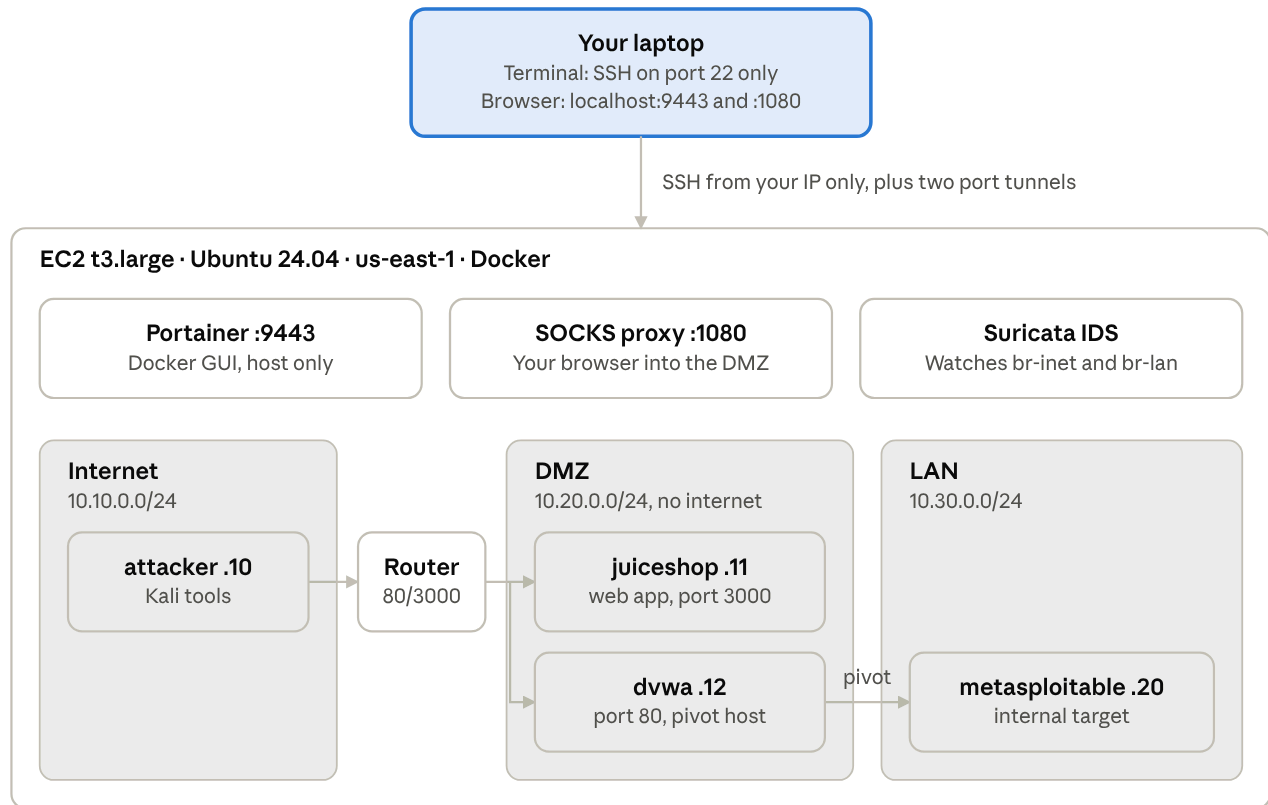
Overview

Each student runs the whole CyberLab, a segmented Internet/DMZ/LAN cyber range with Suricata IDS, on one EC2 t3.large instance inside their own AWS Academy Learner Lab, paid from class credits. The first setup takes about 45-60 minutes, 15 of which is waiting for the automatic install; every later session starts in about 5 minutes.

Who this is for: students in a course that uses AWS Academy Learner Lab, and instructors piloting the lab before class.

How to use this runbook: do Steps 1 to 6 once, in order. Use Step 7 at the start and end of every later session. Instructors should read Instructor notes before assigning the lab.

Students reach the lab only through an SSH tunnel



CyberLab on one Learner Lab EC2 instance · 3 zones, 1 SSH entry point

Nothing in the lab is reachable from the internet: the only open port is SSH from your own IP, and Portainer and the SOCKS proxy reach your laptop through that SSH connection. The attacker reaches the DMZ only through the router, and the LAN only by pivoting through DVWA.

Before you start

You need a Learner Lab seat, a computer with an SSH client, and the kit. Stopped between sessions, the lab uses about \$21 of your \$100 class credit per semester.

Checklist

- ☐ You are enrolled in an AWS Academy course that includes the Learner Lab module (the invitation comes by email from AWS Academy).
- ☐ Your computer runs Windows 10/11, macOS or Linux. RAM does not matter, because the lab runs in the cloud.

- ☐ You have cyberlab-kit.zip from your instructor (LMS or GitHub release).
- ☐ Your network allows outbound SSH on TCP port 22. Some campus guest and hotel Wi-Fi block it; a phone hotspot is a good test.
- ☐ Firefox is installed. Its proxy settings are separate from the rest of your computer, which makes the SOCKS step easy.
- ☐ You have read and accepted your course's acceptable use policy.

Learner Lab rules that shape this setup

Rule	Limit	What it means for CyberLab
Regions	us-east-1 and us-west-2 only	Use us-east-1 (N. Virginia), the only region with the vockey key pair
Instance types	nano to large, On-Demand only	t3.large is the largest allowed; Spot is not available
Instance count	32 vCPU and 9 instances per region	One CyberLab uses 2 vCPU
Images (AMIs)	Amazon-provided Linux and Windows only	Ubuntu 24.04 from Quick Start; the Kali tools come from the attacker container
Disk (EBS)	Up to 100 GB; gp2, gp3, sc1 or standard	40 GB gp3
IAM	No new users or roles; LabRole and LabInstanceProfile exist	Attach LabInstanceProfile to get a browser terminal through Session Manager
Session	Up to 4 hours	End Lab or the timer stops running instances; they restart at the next Start Lab
Budget	\$100 per student per class	Spending all of it deactivates the account and deletes your work; the display lags up to 8 hours
Public IP	Changes at every stop and start	Copy the new IP at the start of each session

What it costs from your credit

Item	Rate	16-week semester, 6 hours a week (96 h)
t3.large compute	\$0.0832 per running hour	\$7.99
Public IPv4 address	\$0.005 per running hour	\$0.48
40 GB gp3 disk	\$3.20 a month, running or stopped	\$12.80 (4 months)
Total, stopped between sessions		about \$21

Left running around the clock, the same instance costs about \$68 a month, which would use the whole \$100 credit in about six weeks. Stopping it after every session (Step 7) is the one habit that matters.

Step 1: Start the Learner Lab session

Start a session, download the vockey private key once, and open the AWS console in us-east-1. Time: about 5 minutes.

1. Go to awsacademy.instructure.com and sign in with the email address that received the AWS Academy invitation.
2. On the Dashboard, open your course, click **Modules**, then **Launch AWS Academy Learner Lab**. The first time, accept the terms of use at the bottom of the page.
3. Click **Start Lab** at the top right of the lab panel. The circle next to **AWS** turns yellow while the session starts and green when it is ready; the first start can take 5 minutes or more.
4. Note the two readouts at the top: the session timer (up to 4:00) and the budget used. Your session ends when the timer reaches zero.
5. Click **AWS Details**. Next to **SSH key**, click **Download PEM** and save the file as `labsuser.pem`. You only do this once: the same key works in every session.
6. Click **AWS** (next to the green circle) to open the AWS Management Console in a new tab. If nothing opens, allow pop-ups for the Academy site.
7. In the console, check the region selector at the top right. It must say **N. Virginia** (us-east-1); if not, choose **US East (N. Virginia)**.

Check: the circle is green, the console shows N. Virginia, and labsuser.pem is in your Downloads folder.

Step 2: Prepare your computer and the kit

Put the key and the kit in one folder, lock down the key's permissions, and check the kit's checksums. Time: about 5 minutes.

Windows 10/11 (PowerShell)

1. Open **PowerShell** (Start menu, type PowerShell). You do not need administrator rights.
2. Check that the built-in SSH client is installed. If `ssh -V` prints an error, go to Settings → System → Optional features → Add a feature → **OpenSSH Client**.
3. Create the folder, move the files and unzip the kit:

```
ssh -V
mkdir $HOME\cyberlab; cd $HOME\cyberlab
Move-Item $HOME\Downloads\labsuser.pem .
Expand-Archive $HOME\Downloads\cyberlab-kit.zip -DestinationPath .
```

4. Restrict the key so only you can read it. SSH refuses a key that other users can read.

```
icacls labsuser.pem /inheritance:r /grant:r "$($env:USERNAME):(R)"
```

5. Check the kit: run `Get-FileHash .\cyberlab-kit\bootstrap-cloud.sh -Algorithm SHA256` and compare the hash with the bootstrap-cloud.sh line in cyberlab-kit\SHA256SUMS. They must match exactly.

macOS or Linux (Terminal)

```
mkdir -p ~/cyberlab && cd ~/cyberlab
mv ~/Downloads/labsuser.pem . && chmod 400 labsuser.pem
unzip ~/Downloads/cyberlab-kit.zip
cd cyberlab-kit
sha256sum -c SHA256SUMS # macOS: shasum -a 256 -c SHA256SUMS
```

Every line should end in OK. If any line says FAILED, download the kit again before continuing.

Open the bootstrap file

Open `cyberlab-kit/bootstrap-cloud.sh` in a plain-text editor: Notepad, VS Code, or TextEdit after Format → Make Plain Text. You will copy all of it in Step 3. Do not use Word: it changes quotes and breaks the script.

Check: your `cyberlab` folder holds `labsuser.pem` and the `cyberlab-kit` folder, and the checksum matches.

Required scripts

Only one file has to reach the instance: `bootstrap-cloud.sh`, which carries the lab and the `cyberlab` manager inside it. The two AWS scripts run on your laptop and are optional.

File in the kit	Runs on	Needed?	What it does
<code>bootstrap-cloud.sh</code>	EC2 instance, as user data (Step 3) or with <code>sudo</code>	Required	Installs Docker, a host firewall (SSH only), fail2ban, automatic updates and swap; writes the lab to <code>/opt/cyberlab</code> , installs the <code>cyberlab</code> command and starts the lab. Safe to run twice
<code>lab/docker-compose.yml</code>	Instance, written there by the bootstrap	Built in	Defines the eight containers and the Internet, DMZ and LAN networks
<code>scripts/cyberlab</code>	Instance, installed by the bootstrap	Built in	Lab manager: <code>up</code> , <code>verify</code> , <code>status</code> , <code>alerts</code> , <code>rules</code> , <code>reset</code> , <code>down</code> , <code>destroy</code>
<code>aws/learner-lab-launch.sh</code>	Your laptop, with the AWS CLI	Optional	Launches the instance in one command (Step 3 alternative)

File in the kit	Runs on	Needed?	What it does
aws/learner-lab-ip.sh	Your laptop, with the AWS CLI	Optional	Starts the stopped instance, allows SSH from your current IP and prints the new address (Step 7)
SHA256SUMS	Your laptop and the instance	Recommended	Checksums that prove no file was changed in transit

Copy the kit to the instance with SSH (scp)

With user data, the bootstrap reaches the instance by itself, so this copy is optional. Use it when you forgot or mistyped the user data, when you want to rerun the bootstrap, or when you want the whole kit on the instance. `scp` copies files over the same SSH connection, key and port 22 as Step 4.

Run these on **your laptop**, not inside an SSH session. Replace `<public-ip>` with this session's address.

```
# Windows PowerShell
cd $HOME\cyberlab
scp -i labsuser.pem $HOME\Downloads\cyberlab-kit.zip ubuntu@<public-ip>:~
```

```
# macOS / Linux
cd ~/cyberlab
scp -i labsuser.pem ~/Downloads/cyberlab-kit.zip ubuntu@<public-ip>:~
```

Then connect (Step 4) and unpack, check and run it on the instance:

```
sudo apt-get update && sudo apt-get install -y unzip
unzip -o cyberlab-kit.zip && cd cyberlab-kit
sha256sum -c SHA256SUMS                # every line should end in OK
sudo bash bootstrap-cloud.sh            # skip if the user data already built
the lab
```

Watch progress with `sudo tail -f /var/log/cyberlab-bootstrap.log` and finish with Step 5.

Other useful copies

Task	Command, run on your laptop
Copy only the bootstrap	<code>scp -i labsuser.pem cyberlab-kit/bootstrap-cloud.sh ubuntu@<public-ip>:~</code>
Copy the unzipped kit folder	<code>scp -i labsuser.pem -r cyberlab-kit ubuntu@<public-ip>:~</code>
Use the SSH config shortcut from Step 4	<code>scp ~/Downloads/cyberlab-kit.zip cyberlab:~</code>
Bring IDS evidence back for a lab report	On the instance: <code>docker cp suricata:/var/log/suricata/eve.json ~/eve.json</code> ; then on the laptop: <code>scp -i labsuser.pem ubuntu@<public-ip>:~/eve.json .</code>

If scp says `Permission denied (publickey)` or times out, the causes and fixes are the same as for SSH in Troubleshooting.

Step 3: Launch the EC2 instance (console)

Launch one t3.large Ubuntu 24.04 instance named cyberlab, with SSH open only to your IP and the bootstrap pasted as user data. Time: about 10 minutes.

Where Ubuntu 24.04 comes from: you do not download or install Ubuntu yourself. On EC2 the operating system arrives preinstalled in the image (AMI) you pick at launch, so choosing **Ubuntu Server 24.04 LTS** in row 2 below *is* the installation. The instance boots already running Ubuntu 24.04, and the bootstrap then installs everything else on top of it. Learner Lab allows only Amazon-provided images, and Canonical's official Ubuntu image in Quick Start is one of them.

In the console, type **EC2** in the search bar, open **EC2**, and click **Launch instance**. Work down the page section by section.

Section	Setting	Value
Name and tags	Name	<code>cyberlab</code> (exactly this; the helper scripts look for it)

Section	Setting	Value
Application and OS Images	Quick Start	Ubuntu
	Image	Ubuntu Server 24.04 LTS (HVM), SSD Volume Type
	Architecture	64-bit (x86) , never Arm: the targets are x86-only
Instance type	Type	t3.large (2 vCPU, 8 GiB)
Key pair (login)	Key pair name	vockey
Network settings	Firewall (security groups)	Create security group
	Allow SSH traffic from	My IP (not Anywhere)
	Allow HTTPS / HTTP from the internet	Both unchecked
Configure storage	Root volume	40 GiB, gp3
Advanced details	IAM instance profile	LabInstanceProfile
	User data	The whole of bootstrap-cloud.sh (below)
Summary	Number of instances	1

The steps in order

1. **Name and tags:** type `cyberlab`.
2. **Application and OS Images:** click the **Ubuntu** tile, then pick **Ubuntu Server 24.04 LTS** from the list. Confirm the architecture shows **64-bit (x86)** and the image says Free tier eligible or Verified provider: Canonical.
3. **Instance type:** open the list and choose **t3.large**. Larger types are blocked in Learner Lab and are terminated if launched.
4. **Key pair:** choose **vockey**. If vockey is missing, you are not in us-east-1; change the region and start again.
5. **Network settings:** leave the default VPC and subnet. Under Firewall, keep **Create security group**, change **Allow SSH traffic from** to **My IP**, and leave HTTPS and HTTP unchecked. Click **Edit** if you want to rename the group to `cyberlab-sg`.

6. **Configure storage:** change **8** to **40** and make sure the type is **gp3**.

7. **Advanced details** (click to expand):

- **IAM instance profile:** choose **LabInstanceProfile**. This lets you open a browser terminal through Session Manager if SSH ever fails.
- **User data:** scroll to the bottom. In your editor, select all of bootstrap-cloud.sh (Ctrl+A or Cmd+A), copy it, and paste it into the box. The first line must be `#!/usr/bin/env bash`. The file is about 14 KB, under the 16 KB limit.
- Leave everything else at its default.

8. **Summary** (right side): check 1 instance, t3.large, 40 GiB, then click **Launch instance**.

9. Click **View all instances**. Select **cyberlab** and wait until **Instance state** is Running and **Status check** shows 2/2 checks passed (2-3 minutes; click refresh).

10. In the details pane below, copy the **Public IPv4 address**. You need it in Step 4.

Check: cyberlab is Running with 2/2 checks, and its security group has one inbound rule: SSH, port 22, from your IP ending in /32 (Security tab → Inbound rules).

The bootstrap starts by itself at first boot. It installs Docker, a host firewall (SSH only), fail2ban and automatic updates, adds swap, downloads the lab images and starts the lab. Continue to Step 4 while it runs.

Step 3 (alternative): Launch with the AWS CLI script

learner-lab-launch.sh does all of Step 3 in one command. Use it instead of the console, not as well as it: each run launches a new instance.

1. Install AWS CLI v2 on your computer (aws.amazon.com/cli). On Windows, run the kit's scripts from Git Bash or WSL, because they are bash scripts.
2. In the Learner Lab panel, click **AWS Details**, then **Show** next to **AWS CLI**. Copy the whole block, starting at `[default]`.
3. Paste it into your credentials file, replacing anything already there:
 - macOS/Linux: `~/.aws/credentials`
 - Windows: `%USERPROFILE%\aws\credentials`
4. Run:

```
cd ~/cyberlab/cyberlab-kit
aws sts get-caller-identity          # prints your lab account; if it errors,
redo step 3
bash aws/learner-lab-launch.sh bootstrap-cloud.sh
```

The script:

- sets the region to us-east-1;
- looks up the current Ubuntu 24.04 x86 AMI from Canonical's public SSM parameter;
- creates the security group cyberlab-sg in the default VPC, with one rule: SSH from your current public IP /32;
- launches a t3.large with vockey, LabInstanceProfile, a 40 GB gp3 disk, the bootstrap as user data and the Name tag cyberlab;
- waits until the instance is running, then prints its public IP and the exact SSH command for Step 4.

The credentials expire when the session ends. Paste the new block from AWS Details at the start of every session in which you use the CLI.

Do not run the script in AWS CloudShell. It reads "your IP" from the machine it runs on, so in CloudShell the SSH rule would allow CloudShell's address instead of yours.

Step 4: Connect over SSH with the two tunnels

One SSH command logs you in as ubuntu and carries two tunnels: port 9443 for Portainer and port 1080 for the SOCKS proxy. Keep this window open for as long as you use the lab.

Replace `<public-ip>` with the address from Step 3.

```
# Windows PowerShell
cd $HOME\cyberlab
ssh -i labsuser.pem -L 9443:127.0.0.1:9443 -L 1080:127.0.0.1:1080
ubuntu@<public-ip>
```

```
# macOS / Linux
cd ~/cyberlab
ssh -i labsuser.pem -L 9443:127.0.0.1:9443 -L 1080:127.0.0.1:1080
ubuntu@<public-ip>
```

1. The first time, SSH asks whether you trust the host's fingerprint. Type `yes` and press Enter.

2. You are in when the prompt reads `ubuntu@ip-172-31-...:~$`.
3. If you get "Connection refused" in the first 2 minutes, the instance is still booting; wait a minute and retry.

Part of the command	Meaning
<code>-i labsuser.pem</code>	Log in with the vockey private key
<code>-L 9443:127.0.0.1:9443</code>	Your laptop's localhost:9443 reaches Portainer on the instance
<code>-L 1080:127.0.0.1:1080</code>	Your laptop's localhost:1080 reaches the lab's SOCKS proxy
<code>ubuntu@<public-ip></code>	Ubuntu's default user on this instance

Optional: a shortcut in your SSH config

Add this to `~/.ssh/config` (Windows: `%USERPROFILE%\.ssh\config`), then connect with just `ssh cyberlab`. Update the `HostName` line each session, because the IP changes.

```
Host cyberlab
  HostName <public-ip>
  User ubuntu
  IdentityFile ~/cyberlab/labsuser.pem
  LocalForward 9443 127.0.0.1:9443
  LocalForward 1080 127.0.0.1:1080
  ServerAliveInterval 60
```

Backup: a browser terminal

If SSH is blocked on your network, select cyberlab in EC2 → **Connect** → **Session Manager** tab → **Connect**. This works because of `LabInstanceProfile`. Type `sudo -iu ubuntu` first. It gives you a shell for commands but no tunnels, so Portainer and the SOCKS proxy are unavailable in this mode.

Step 5: Watch the bootstrap and verify the lab

The lab is ready when the bootstrap log says done and `cyberlab verify` shows PASS on every check. Time: 10-15 minutes, mostly waiting.

1. In your SSH window, follow the install log:

```
sudo tail -f /var/log/cyberlab-bootstrap.log
```

2. Wait for the final line saying the bootstrap is done, then press **Ctrl+C** to stop following the log. The longest part is downloading about 3 GB of images and installing the attacker's Kali tools.

3. Confirm that first-boot setup finished cleanly. It should print `status: done`; `status: error` means read the log (Troubleshooting).

```
cloud-init status --wait
```

4. Type `exit`, then reconnect with the Step 4 command. Logging in again gives the `ubuntu` user its Docker permissions.

5. Run the self-test and the status view:

```
cyberlab verify
cyberlab status
```

6. `cyberlab verify` must show **PASS** on every line. `cyberlab status` should list eight running containers: `attacker`, `router`, `juiceshop`, `dvwa`, `metasploitable`, `suricata`, `portainer` and `socks`.

To confirm the instance runs the right operating system, run `lsb_release -d`. It should print `Ubuntu 24.04` followed by the point release (for example `24.04.3 LTS`). Any other version means the wrong image was picked in Step 3: terminate this instance and launch again.

Optional: save a clean snapshot

Once everything passes, take a disk snapshot as a restore point for an operating system that gets damaged later. The steps, the cost (about \$0.50 a month) and how to restore are in **Back up and restore** at the end of this guide. `cyberlab reset` (Step 6) covers most lab-level repairs without one.

Step 6: First lab session walkthrough

In this first session you set up Portainer, scan the DMZ from the attacker, open the web targets in Firefox and watch Suricata catch your scan. Time: about 30 minutes.

A. Set up Portainer

1. With the Step 4 SSH window open, browse on your laptop to **https://localhost:9443**.
2. The certificate is self-signed, so the browser warns you. Click **Advanced** → **Accept the risk and continue** (Firefox) or **Proceed to localhost** (Chrome).
3. Create the admin user with a password of 12 characters or more. Do this within 5 minutes of the lab starting; if the page says the instance timed out, run `docker restart portainer` over SSH and reload.
4. Click **Get Started** to manage the local Docker environment. You now see every lab container, its logs and its console.

B. Scan the DMZ from the attacker

```
docker exec -it attacker bash      # you are now inside the Kali container
ip -brief addr                    # shows 10.10.0.10 on the Internet
network
nmap -sV 10.20.0.0/24             # finds juiceshop .11:3000 and dvwa
                                  # .12:80
ping -c 2 10.30.0.20             # fails: the LAN is not reachable from
the Internet zone
exit
```

C. Browse the targets through the SOCKS proxy

1. In Firefox, open **Settings** → **General** → **Network Settings** → **Settings....**
2. Choose **Manual proxy configuration**. Leave HTTP Proxy empty. Set **SOCKS Host** to `127.0.0.1`, **Port** to `1080`, and select **SOCKS v5**. Click **OK**.
3. Open **http://10.20.0.11:3000** (OWASP Juice Shop) and **http://10.20.0.12** (DVWA). Use IP addresses, not names.
4. For DVWA, log in with the defaults admin / password, click **Create / Reset Database**, log in again, then set **DVWA Security** to Low for the first exercises.
5. When you finish, set Firefox back to **No proxy**. A separate Firefox profile for the lab (about:profiles) saves switching every time.

D. Watch the defender's view

1. Open a second terminal and connect without the tunnels: `ssh -i labsuser.pem ubuntu@<public-ip>`.

2. Run `cyberlab alerts` there.
3. Repeat the nmap scan in the first window. Suricata scan alerts appear in the second window within seconds. Press Ctrl+C to stop.

E. Everyday lab commands

Task	Command
Attack shell	<code>docker exec -it attacker bash</code>
Lab status	<code>cyberlab status</code>
Live IDS alerts	<code>cyberlab alerts</code>
Full IDS events (JSON)	<code>docker exec suricata tail -f /var/log/suricata/eve.json</code>
Update Suricata rules	<code>cyberlab rules</code>
Reset the targets, keep rules and logs	<code>cyberlab reset</code>
Reset everything	<code>cyberlab reset --full</code>
Stop the lab containers	<code>cyberlab down</code>

Suggested first exercise: reconnaissance with nmap, DVWA command injection, a pivot from DVWA to Metasploitable on 10.30.0.20, then switch roles and find each step in `cyberlab alerts`. Ask why the DMZ bridge is not monitored: the router's NAT hides the attacker's address there.

Step 7: End each session, and start the next one

Stop the instance yourself at the end of every session, then End Lab; next time, start it, copy the new IP and reconnect. Each takes about 5 minutes.

Ending a session

1. Type `exit` in each SSH window.
2. In EC2, select **cyberlab** → **Instance state** → **Stop instance** → **Stop**. Never choose **Terminate**: it deletes the disk and all your work.
3. Wait until the state shows **Stopped**.

4. In the Learner Lab panel, click **End Lab** and confirm.

Why stop it yourself: End Lab stops running instances, but Learner Lab restarts them at your next Start Lab, even if you only opened the lab to check your budget. An instance you stopped stays stopped until you start it.

If your 4-hour timer is running low mid-lab, click **Start Lab** again to reset it.

Starting the next session

1. **Start Lab** and wait for the green circle, then open the console (**AWS**).
2. In EC2, select **cyberlab**. If it shows Stopped, choose **Instance state** → **Start instance** and wait for Running.
3. Copy the new **Public IPv4 address**. It changes after every stop.
4. If you are on a different network than last time (home vs. campus), update the SSH rule: **Security** tab → click the security group → **Edit inbound rules** → SSH row → Source **My IP** → **Save rules**.
5. Connect with the Step 4 command and the new IP. The containers restart by themselves; wait 2 minutes, then run `cyberlab verify`.

Faster: the restart script

With fresh credentials from AWS Details in your credentials file (Step 3 alternative), one command starts the instance, allows SSH from your current IP and prints the SSH command:

```
cd ~/cyberlab/cyberlab-kit
bash aws/learner-lab-ip.sh
```

The script adds a rule for each new IP but never removes old ones. Once in a while, delete the SSH rules for addresses you no longer use.

Instructor notes

Pilot the whole runbook on your own seat, set End Lab to stop rather than terminate, and check class spend weekly. A 30-student class uses about \$630 of AWS Academy credit per semester and nothing from the department.

Before the term

- ☐ Run Steps 1 to 6 yourself on a Learner Lab seat. The scripts pass syntax and lint checks but have not been run end to end on every setup.
- ☐ In the Learner Lab's budget settings, set the **End Lab** behavior to stop resources, not terminate them. Terminate would delete every student's lab at the end of each session.
- ☐ Set the class end date to cover the whole term, including finals.
- ☐ Pin the image versions in lab/docker-compose.yml (replace `:latest` with the tags you tested), rebuild the bootstrap with build/build-bootstrap.py, and regenerate SHA256SUMS.
- ☐ Post cyberlab-kit.zip and SHA256SUMS on the LMS with this runbook.
- ☐ Ask campus IT to allow outbound SSH (TCP 22) to AWS from lab rooms and campus Wi-Fi.
- ☐ Publish the acceptable use policy and have students accept it before they get the kit.

Running the first lab

- Assign Steps 1 to 5 as homework before the first class. The downloads take 15 minutes, and a room of 30 simultaneous installs is slow.
- Start class with Step 6. Students whose bootstrap failed can run `cyberlab verify` and read the Troubleshooting table while you help others.
- Keep a spare instance of your own running for demonstrations and for students who get stuck.

During the term

- Check each student's spend in the Learner Lab analytics once a week. Budget figures lag up to 8 hours.
- Contact any student who has used more than \$50 by mid-term: they are almost certainly leaving the instance running.
- For a student who is stuck, open their workspace from the Learner Lab educator view instead of asking for screenshots.

End of term

- Have students terminate cyberlab and delete any images and snapshots they created, so no vulnerable host keeps running.

- Collect lab reports before the class end date; after it, students lose access to their accounts.

Mapping to NICE work roles

Lab activity	NICE Framework work role
Reconnaissance and web testing (nmap, Juice Shop, DVWA)	Vulnerability Analysis
Router firewall policy and network segmentation	Cybersecurity Infrastructure Support
Suricata alerts and attack timelines	Defensive Cybersecurity; Incident Response
Docker host hardening and resets	Systems Administration

Confirm role names and TKS statements against the current NICE Framework release before citing them in a syllabus.

Troubleshooting

Most problems are a changed IP address, a key file with loose permissions, or the attacker's first-start install still running.

Symptom	Likely cause	Fix
vockey is missing in the key pair list	Console is not in us-east-1	Switch the region to N. Virginia and start the launch again
Launch fails with an instance type or AMI error	Type larger than large, or a Marketplace image	Use t3.large and Ubuntu 24.04 from Quick Start
SSH: Connection timed out	Your public IP changed, or the network blocks port 22	Edit the SSH rule to My IP (Step 7); try a phone hotspot
SSH: Connection refused	Instance still booting	Wait 1-2 minutes after 2/2 checks pass, then retry

Symptom	Likely cause	Fix
SSH: UNPROTECTED PRIVATE KEY FILE or bad permissions	Key readable by other users	Rerun the chmod or icacIs command from Step 2
SSH: Permission denied (publickey)	Wrong user or wrong key	Log in as ubuntu with labsuser.pem
SSH: REMOTE HOST IDENTIFICATION HAS CHANGED	The IP now belongs to a new instance	<code>ssh-keygen -R <public-ip></code> , then connect again
<code>bind</code> : Address already in use for 9443 or 1080	A second tunnel window is open	Close the other SSH window, or connect the second one without <code>-L</code>
Bootstrap log stops or <code>cloud-init status</code> says error	Download or apt failure	Read <code>/var/log/cyberlab-bootstrap.log</code> , then rerun <code>sudo bash /var/lib/cloud/instance/scripts/part-001</code> ; it is safe to run twice
<code>docker</code> : permission denied	Session started before the bootstrap added you to the docker group	<code>exit</code> and reconnect
<code>cyberlab up</code> waits a long time on attacker	First start installs the Kali tools	Wait up to 15 minutes; watch <code>docker logs -f attacker</code>
Containers exit with code 137	Out of memory	<code>docker stop portainer juiceshop</code> for lighter exercises
Suricata shows no alerts	Rules not loaded yet	<code>cyberlab rules</code> , then scan again
Suricata exits at start	Lab bridges did not exist yet	<code>docker restart suricata</code> after the lab is up
Firefox shows no page through the proxy	Tunnel closed, or a name used instead of an IP	Reconnect with the Step 4 command; use 10.20.0.11 and 10.20.0.12
SOCKS stops after the attacker restarts	It shares the attacker's network	<code>docker restart socks</code>

Symptom	Likely cause	Fix
Portainer says the instance timed out	Admin user not created within 5 minutes	<code>docker restart portainer</code> , then reload
Instance is running though you ended the lab	Learner Lab restarted it at Start Lab	Stop it yourself before End Lab (Step 7)

Safety rules

The lab contains intentionally vulnerable software, so it stays reachable only by you and only through SSH.

1. Never add inbound rules other than SSH from your own IP. Never open HTTP, HTTPS, 3000, 9443 or 1080 in the security group.
2. Keep Portainer and the SOCKS proxy bound to 127.0.0.1, as the kit ships them. Portainer controls Docker, which means root on the instance.
3. Point lab tools only at 10.10.0.0/24, 10.20.0.0/24 and 10.30.0.0/24. Scanning anything else, including classmates' instances or AWS addresses, breaks the acceptable use policy and may break the law.
4. Run exploit tools and Atomic Red Team tests only inside the attacker container, never on the instance itself.
5. Stop the instance after every session, and terminate it when the course ends.

Back up and restore

Back up two things separately: a **disk snapshot** rolls back a damaged operating system, and a **copy of your own work** survives even if the instance is lost. The lab itself is disposable, because the containers and even the whole host can be rebuilt from the kit.

1. Snapshot the disk once the lab passes `cyberlab verify`

1. Stop the instance so the snapshot is clean: EC2 → select **cyberlab** → **Instance state** → **Stop**.
2. Open the **Storage** tab, click the volume ID, then **Actions** → **Create snapshot**.

3. Add a description such as `cyberlab-clean-2026-09-27` and click **Create snapshot**. It completes in a few minutes; you can start the instance again straight away.

Take one snapshot after setup and another after any milestone worth keeping. Snapshots store only used blocks, at about \$0.05 per GB-month: about \$0.50 a month for this lab's roughly 10 GB. Delete old snapshots (EC2 → **Snapshots**) so they do not accumulate against your \$100 credit.

Restore a snapshot: EC2 → select the instance → **Actions** → **Monitor and troubleshoot** → **Replace root volume** → choose the snapshot → **Create**. The instance keeps its ID and security group and reboots onto the saved disk.

If Learner Lab refuses Replace root volume, restore by hand:

1. EC2 → **Snapshots** → select the snapshot → **Actions** → **Create volume from snapshot**, in the same Availability Zone as the instance (shown on its Networking tab).
2. Stop the instance. On its **Storage** tab, note the root device name (usually `/dev/sda1`), then detach the old root volume (**Volumes** → **Actions** → **Detach volume**).
3. Attach the new volume to the instance with the same device name, then start the instance.

Prefer snapshots to "Create image" (an AMI): Learner Lab allows launches from Amazon-provided AMIs only, so launching from your own image may be refused. Instructors should test one full restore on their own seat before students rely on it.

2. Copy your work off the instance

Keep lab notes, scripts and Suricata evidence where a broken disk cannot reach them. Put your files in `~/work` as you go.

```
# on the instance: gather your work into one dated archive
mkdir -p ~/work
docker cp suricata:/var/log/suricata/eve.json ~/work/
tar czf ~/work-$(date +%F).tgz -C ~ work
```

```
# on your laptop: pull the archive down
scp -i labsuser.pem ubuntu@<public-ip>:~/work-*.tgz .
```

Or keep it in S3, which the instance reaches through LabInstanceProfile. S3 storage for a few archives costs pennies a month.

```

sudo snap install aws-cli --classic                # once, if the aws
command is missing
aws s3 mb s3://cyberlab-<yourname>-work            # once; bucket names must
be globally unique
aws s3 cp ~/work-$(date +%F).tgz s3://cyberlab-<yourname>-work/
aws s3 cp s3://cyberlab-<yourname>-work/<archive>.tgz ~/ # to restore

```

3. Match the fix to the damage

What broke	Fastest fix
A target or container misbehaves	<code>cyberlab reset</code>
Lab files, rules or Docker setup changed	<code>cyberlab reset --full</code> , or rerun <code>sudo bash bootstrap-cloud.sh</code>
OS files deleted or changed, SSH fails, the instance will not boot	Restore the disk snapshot (section 1)
OS broken and no snapshot	Terminate, launch a new instance with the user data (Step 3, about 15 minutes), then copy your work back
Your own files lost	Restore the archive from your laptop or S3 (section 2)

Working from a public or locked-down computer

On a library or lab PC where you cannot install software, and port 22 may be blocked, do everything in the web browser: Session Manager gives you a terminal, and Termshark replaces the Wireshark window.

1. Open a terminal in the browser with Session Manager

Session Manager needs no SSH client, no key file and no port 22; it runs over ordinary HTTPS.

1. **Start Lab**, open the console (**AWS**), then EC2 → select **cyberlab** → **Connect** → **Session Manager** tab → **Connect**.
2. Type `sudo -iu ubuntu` to switch to the usual lab user.

This works because the instance has `LabInstanceProfile` attached (Step 3). Every `cyberlab` and `docker` command in this guide works here.

2. Use Termshark instead of the Wireshark window

Termshark is a Wireshark-style interface (packet list, details and hex panes, display filters) that runs inside a terminal, so it works in the Session Manager tab.

```
sudo DEBIAN_FRONTEND=noninteractive apt-get install -y termshark # once;
also installs tshark
sudo termshark -i br-inet # live capture of attacker ↔
DMZ traffic
termshark -r ~/work/scan.pcap # open a saved capture
```

Filters use Wireshark syntax, for example `http.request` or `tcp.flags.syn==1 && tcp.flags.ack==0`. Press `/` to jump to the filter bar and `q` to quit. List the lab's capture interfaces with `ip -br link | grep br-`.

The same engine, tshark, answers most analysis questions directly:

```
tshark -r scan.pcap -q -z io,phs # protocol hierarchy
tshark -r scan.pcap -q -z conv,tcp # conversations
tshark -r scan.pcap -q -z follow,tcp,ascii,0 # follow TCP stream 0
```

3. If the assignment needs the real Wireshark window

Run Wireshark as a container that serves its GUI as a web page over HTTPS, protected by a password and open only to the library's address:

```
docker run -d --name wireshark --network host --cap-add NET_ADMIN \
  -e PUID=1000 -e PGID=1000 -e CUSTOM_USER=student -e PASSWORD='<long-random-
  password>' \
  lscr.io/linuxserver/wireshark:latest
sudo ufw allow from <library-public-ip> to any port 3001 proto tcp
```

1. In the instance's security group, add an inbound rule: **Custom TCP**, port **3001**, source **My IP**.
2. In the library browser, open **https://<public-ip>:3001**, accept the certificate warning and sign in.
3. When finished, remove access again:

```
docker stop wireshark
sudo ufw delete allow from <library-public-ip> to any port 3001 proto tcp
```

Then delete the port 3001 rule from the security group.

Everyone in the library shares the same public IP, so the password is the only real barrier: use this only when the GUI is required, and close it right after. The container uses about 500 MB of RAM. For whole classes, an instructor-run Guacamole server is the better answer: students get desktops and terminals in any browser, and nothing is opened on their lab instances.

What does not work from a public computer

Feature	Why it fails	Workaround
Portainer (localhost:9443)	Needs the SSH tunnel	<code>cyberlab status</code> and <code>docker logs <name></code>
Browsing DVWA and Juice Shop	Needs the SOCKS tunnel	<code>curl</code> from the attacker container, or do the web exercises on a personal device
scp copies to the computer	No SSH client, and files should not stay on a shared PC	Save work to S3 (Back up and restore, section 2)

Public computer hygiene

- Never download labsuser.pem to a shared computer; Session Manager does not need it.
- When finished: stop the instance, click **End Lab**, sign out of AWS Academy and close every browser window.

Sources

- [AWS Academy Learner Lab – Foundation Services](#) (regions, instance limits, AMIs, EBS, vockey, LabInstanceProfile)
- [AWS Academy Learner Lab Student Guide](#) (End Lab, restart behavior, \$100 budget)
- [AWS Learner Lab tutorial](#) (4-hour sessions, End Lab setting for educators)

- [SpendArk – t3.large pricing](#) (\$0.0832 per hour in us-east-1)
- CyberLab Deployment Guide, 26 September 2026 (the kit, scripts and lab design this runbook uses)

Prices and Learner Lab limits were checked on 27 September 2026 and can change.